Document number	P2955R1
Date	2023-9-2
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	SG23 Safety and Security Library Evolution Working Group (LEWG)

Safer Range Access

Table of contents

- Safer Range Access
  - Changelog
  - Abstract
  - Technical Details
    - The common challenges
    - The common safe member functions
    - The common safe free functions
  - Optional Design Considerations
  - Example
  - Resolution
  - Summary
  - Frequently Asked Questions
    - Why not just wait for contracts
  - References

Changelog

R1

- Minor verbiage changes

Abstract

Currently the element access of `std::array`, `std::deque`, `std::span`, `std::string`, `std::string_view`, `std::vector` is not as safe as it could be. While the `at` member functions are checked for range access errors, the `operator[]`, `front` and `back` are not checked. That means the majority of the element access is unchecked. Further, even though the rarely used `at` is checked, like the others, it returns a reference which, besides exposing implementation details, can lead to dangling and reference invalidation errors.

This paper proposes that we provide programmers with functions that have value semantics in order to not only mitigate range access errors but also to reduce the dangling and reference invalidation errors.

Technical Details

Feel free to jump to the end for the example code.

These technical details will focus on `vector` but everything can be easily applied to the other containers and views with range based access. The first method to be added is a `test` function since only a given implementation can say whether an index is valid.

```
[[nodiscard]]
[[safe]]
constexpr bool test( std::vector<T>::size_type pos ) const;
```

The next two methods requested for addition are actually `unsafe`. Though `unsafe`, these will be the foundation for which all of the `safe` functions will be built upon.

```
[[nodiscard]]
[[unsafe(reason=["range", "dangling_reference", "reference_invalidation"])]]
constexpr reference get_reference( size_type pos ) noexcept;
[[nodiscard]]
[[unsafe(reason=["range", "dangling_reference", "reference_invalidation"])]]
constexpr const_reference get_reference( size_type pos ) const noexcept;
```

It should be noted that these functions are just as `unsafe` as the current subscript operators. The difference is while the subscript operators has undefined unsafety, the proposed `get_reference` deliberately perform no checks as the checks will be performed either by the programmer or by `safe` functions that call these `unsafe` ones. This way when `contracts` arrive and when the undefined behavior is changed to defined behavior, the `safe` functions won’t be performing an additional superfluous test.

```
[[nodiscard]]
[[unsafe(reason=["range", "dangling_reference", "reference_invalidation"])]]
constexpr reference operator[]( size_type pos );
[[nodiscard]]
[[unsafe(reason=["range", "dangling_reference", "reference_invalidation"])]]
constexpr const_reference operator[]( size_type pos ) const;
// ...
[[nodiscard]]
[[unsafe(reason=["dangling_reference", "reference_invalidation"])]]
constexpr reference at( size_type pos );
[[nodiscard]]
[[unsafe(reason=["dangling_reference", "reference_invalidation"])]]
constexpr const_reference at( size_type pos ) const;
```

The common challenges

1. `C++` 's subscript operators tend to have reference semantics instead of value semantics because our language does not have seperate operators for getting and setting values.
    1. `C++` 's subscript operators does not support additional parameters such as `std::source_location` , assigned value or default value which are not part of the indexer.
2. `std::optional` , `std::variant` and `std::expected` does not support references so they can not be used simply with the subscript operator

Consequently, this proposal just use functions instead of any operators or transient proxy classes. While I prefer `deducing this` [1] style member functions since these proposed functions won't be overloaded, as most compilers currently do not support such feature, the examples will be regular member functions.

The common safe member functions

Following are the function names of all the requested member functions. They are represented in table fashion to better illustrate the relationships between all of these variants of getters and setters.

1. `operator[]` / `at` , `front` and `back` get seperate getter and setter functions that take and return values.
2. The getter and setter variants gets multiplied by how programmers handle errors.
    - throw exception
    - return default
    - return optional
    - terminate

- do nothing i.e. void i.e. crop
- grow

		front	back
get	get_value	get_front_value	get_back_value
	get_optional	get_front_optional	get_back_optional
	get_or_terminate	get_front_or_terminate	get_back_or_terminate
set	set_value	set_front_value	set_back_value
	set_or_terminate	set_front_or_terminate	set_back_or_terminate
	set_and_crop	set_front_and_crop	set_back_and_crop
	set_and_grow	set_front_and_grow	set_back_and_grow

Returning references don't just allow the programmer to get and set the whole value but also modify a part of the indexed value or call some member function. In order to preserve that capability, the previous get/set table gets duplicated with a transform/visit versions. Like the value based get's above, the transform member functions returns a subset of the index value as a value using a transformer object. Like the value based set's above, the visit member functions are safe because they deal with values and always return void instead of a reference. Its provided copier objects performs a programmer decided action to the indexed value.

		front	back
transform	transform_value	transform_front_value	transform_back_valu
	transform_optional	transform_front_optional	transform_back_opt
	transform_or_terminate	transform_front_or_terminate	transform_back_or_
visit	visit_value	visit_front_value	visit_back_value
	visit_or_terminate	visit_front_or_terminate	visit_back_or_termi
	visit_and_crop	visit_front_and_crop	visit_back_and_crop
	visit_and_grow	visit_front_and_grow	visit_back_and_grow

It would be preferable that all of these member functions in these two tables were actually free functions, if and only if we get the `pipe redirect operator` [2] [3]. This would save on have to specify all of the functions repeatedly on the `std::array`, `std::deque`, `std::span`,

`std::string`, `std::string_view`, `std::vector` classes. The complete list of these member functions from the perspective of `std::vector` follows.

```
// at and operator[] replacements

[[nodiscard]]
[[safe]]
constexpr value_type get_value( size_type pos, const std::source_location location

template<class F>
[[nodiscard]]
[[safe]]
constexpr auto transform_value( size_type pos, const F& transformer, const std::so

[[nodiscard]]
[[safe]]
constexpr value_type get_value( size_type pos, const_reference default_value ) con

template<class F>
[[nodiscard]]
[[safe]]
constexpr auto transform_value( size_type pos, const F& transformer, const auto& d

[[nodiscard]]
[[safe]]
constexpr std::optional<value_type> get_optional( size_type pos ) const noexcept;

template<class F>
[[nodiscard]]
[[safe]]
constexpr auto transform_optional( size_type pos, const F& transformer ) const noe

[[nodiscard]]
[[safe]]
constexpr value_type get_or_terminate( size_type pos, const std::source_location l

template<class F>
[[nodiscard]]
[[safe]]
constexpr auto transform_or_terminate( size_type pos, const F& transformer, const

[[safe]]
constexpr void set_value( size_type pos, const_reference value, const std::source_

template<class F>
[[safe]]
constexpr void visit_value( size_type pos, const F& copier, const std::source_loca

[[safe]]
constexpr void set_or_terminate( size_type pos, const_reference value, const std::

template<class F>
```

```
[[safe]]
constexpr void visit_or_terminate( size_type pos, const F& copier, const std::sour

[[safe]]
constexpr void set_and_crop( size_type pos, const_reference value ) noexcept;

template<class F>
[[safe]]
constexpr void visit_and_crop( size_type pos, const F& copier ) noexcept;

[[safe]]
constexpr void set_and_grow( size_type pos, const_reference value );

template<class F>
[[safe]]
constexpr void visit_and_grow( size_type pos, const F& copier );

[[safe]]
constexpr void set_and_grow( size_type pos, const_reference value, const_reference

template<class F>
[[safe]]
constexpr void visit_and_grow( size_type pos, const F& copier, const_reference def

// front replacements

[[nodiscard]]
[[safe]]
constexpr value_type get_front_value( const std::source_location location = std::s

template<class F>
[[nodiscard]]
[[safe]]
constexpr auto transform_front_value( const F& transformer, const std::source_loca

[[nodiscard]]
[[safe]]
constexpr value_type get_front_value( const_reference default_value ) const noexce

template<class F>
[[nodiscard]]
[[safe]]
constexpr auto transform_front_value( const F& transformer, const auto& default_va

[[nodiscard]]
[[safe]]
constexpr std::optional<value_type> get_front_optional() const noexcept;

template<class F>
```

```
[[nodiscard]]
[[safe]]
constexpr auto transform_front_optional( const F& transformer ) const noexcept;

[[nodiscard]]
[[safe]]
constexpr value_type get_front_or_terminate( const std::source_location location =

template<class F>
[[nodiscard]]
[[safe]]
constexpr auto transform_front_or_terminate( const F& transformer, const std::sour

[[safe]]
constexpr void set_front_value( const_reference value, const std::source_location

template<class F>
[[safe]]
constexpr void visit_front_value( const F& copier, const std::source_location loca

[[safe]]
constexpr void set_front_or_terminate( const_reference value, const std::source_lo

template<class F>
[[safe]]
constexpr void visit_front_or_terminate( const F& copier, const std::source_locati

[[safe]]
constexpr void set_front_and_crop( const_reference value ) noexcept;

template<class F>
[[safe]]
constexpr void visit_front_and_crop( const F& copier ) noexcept;

[[safe]]
constexpr void set_front_and_grow( const_reference value );

template<class F>
[[safe]]
constexpr void visit_front_and_grow( const F& copier );

[[safe]]
constexpr void set_front_and_grow( const_reference value, const_reference default_

template<class F>
[[safe]]
constexpr void visit_front_and_grow( const F& copier, const_reference default_valu

// back replacements
```

```
[[nodiscard]]
[[safe]]
constexpr value_type get_back_value( const std::source_location location = std::so

template<class F>
[[nodiscard]]
[[safe]]
constexpr auto transform_back_value( const F& transformer, const std::source_locat

[[nodiscard]]
[[safe]]
constexpr value_type get_back_value( const_reference default_value ) const noexcep

template<class F>
[[nodiscard]]
[[safe]]
constexpr auto transform_back_value( const F& transformer, const auto& default_val

[[nodiscard]]
[[safe]]
constexpr std::optional<value_type> get_back_optional() const noexcept;

template<class F>
[[nodiscard]]
[[safe]]
constexpr auto transform_back_optional( const F& transformer ) const noexcept;

[[nodiscard]]
[[safe]]
constexpr value_type get_back_or_terminate( const std::source_location location =

template<class F>
[[nodiscard]]
[[safe]]
constexpr auto transform_back_or_terminate( const F& transformer, const std::sourc

[[safe]]
constexpr void set_back_value( const_reference value, const std::source_location l

template<class F>
[[safe]]
constexpr void visit_back_value( const F& copier, const std::source_location locat

[[safe]]
constexpr void set_back_or_terminate( const_reference value, const std::source_loc

template<class F>
[[safe]]
constexpr void visit_back_or_terminate( const F& copier, const std::source_locatio
```

```
  [[safe]]
  constexpr void set_back_and_crop( const_reference value ) noexcept;

  template<class F>
  [[safe]]
  constexpr void visit_back_and_crop( const F& copier ) noexcept;

  [[safe]]
  constexpr void set_back_and_grow( const_reference value );

  template<class F>
  [[safe]]
  constexpr void visit_back_and_grow( const F& copier );

  [[safe]]
  constexpr void set_back_and_grow( const_reference value, const_reference default_v

  template<class F>
  [[safe]]
  constexpr void visit_back_and_grow( const F& copier, const_reference default_value
```

The common safe free functions

Besides these member functions are a couple of safe free functions.

```
  template<class T>
  constexpr void copy_and_crop( std::span<T> destination, std::size_t pos, std::span

  template<class T>
  constexpr void copy_and_grow( std::vector<T>& destination, std::size_t pos, std::s
```

Besides being safe for not returning a reference, these two functions perform batch copies, reducing the number of individual checks that have to be performed. Both batch copy a source span to a destination range based collection or view. In the case of `copy_and_crop`, any elements that are outside the range of the destination do not get copied. In the case of `copy_and_grow`, the destination range based collection grows to accommodate the elements of the source span that exceeds the destinations current bounds. These two functions are analogous to combining two dimensional arrays such as images.

Optional Design Considerations

One of the criticisms of the checked `at` methods is that the `out_of_range` and other exceptions implies a dynamic allocation upon construction of the exception with a custom descriptive message.

This issue could be resolved by adding a new `virtual what` method to `std::exception` that instead of returning a `const char *`, rather it takes and returns a `std::ostream`.

```
class exception : virtual public std::exception {
public:
    virtual std::ostream& what(std::ostream& os) const noexcept
    {
        return os << std::exception::what();
    }
};
```

New `out_of_range` exceptions could be created that don't initially take a `std::string`.

```
class logic_error_v2 : virtual public std::exception {
};

class out_of_range_v2 : virtual public logic_error_v2 {
};

template<class T>
class out_of_range_v2_1 : virtual public out_of_range_v2 {
private:
    const std::vector<T>::size_type pos;
    const std::source_location location;
public:
    out_of_range_v2_1(const std::vector<T>::size_type pos, const std::source_locat
    {
    }
    virtual std::ostream& what(std::ostream& os) const noexcept override
    {
        //os << this->what();
        return os << "file: "
            << location.file_name() << '('
            << location.line() << ':'
            << location.column() << ") `"
            << location.function_name() << "`: "
            << "invalid index: " << pos << '\n';
    }
};
```

This delays creation of the error message to point of use instead of exception creation. While getting the message into a `std::string` is still possible with `std::stringstream`, the end result is no dynamic allocation for `std::string` needs be created, if it was logged instead or sent to a static buffer.

Example

```
#include <string>
#include <vector>
#include <iostream>
#include <source_location>
#include <optional>
#include <algorithm>
#include <span>
#include <cassert>
#include <sstream>

using namespace std::literals::string_literals;

class exception_v2 : virtual public std::exception {
public:
    virtual std::ostream& what(std::ostream& os) const noexcept
    {
        return os << std::exception::what();
    }
};

class logic_error_v2 : virtual public exception_v2 {
};

class out_of_range_v2 : virtual public logic_error_v2 {
};

template<class T>
class out_of_range_v2_1 : virtual public out_of_range_v2 {
private:
    const T pos;
    const std::source_location location;
public:
    out_of_range_v2_1(const T pos, const std::source_location location) : pos{pos}
    {
    }
    virtual std::ostream& what(std::ostream& os) const noexcept override
    {
        //os << this->what();
        return os << "file: "
            << location.file_name() << '('
            << location.line() << ':'
            << location.column() << ") `"
            << location.function_name() << "`: "
            << "invalid index: " << pos << '\n';
    }
};
```

```
// currently 12 unsafe element access methods
// proposed  14 unsafe element access methods, 56 safe element access methods/func
template<class T>
class my_vector : public std::vector<T> {
private:
/*
    static std::string to_string(const std::vector<T>::size_type pos, const std::s
    {
        std::stringstream ss;
        ss << "file: "
            << location.file_name() << '('
            << location.line() << ':'
            << location.column() << ") `"
            << location.function_name() << "`: "
            << "invalid index: " << pos << '\n';
        return ss.str();
    }
*/
    static void log(const std::vector<T>::size_type pos, const std::source_locatio
    {
        /*
        std::clog << "file: "
            << location.file_name() << '('
            << location.line() << ':'
            << location.column() << ") `"
            << location.function_name() << "`: "
            << "invalid index: " << pos << '\n';
        */
        out_of_range_v2_1<size_type> oor{pos, location};
        oor.what(std::clog) << '\n';
    }
public:
    using std::vector<T>::vector;
    typedef std::vector<T>::value_type value_type;
    typedef std::vector<T>::size_type size_type;
    typedef std::vector<T>::reference reference;
    typedef std::vector<T>::const_reference const_reference;

    [[nodiscard]]
    //[[safe]]
    constexpr bool test( std::vector<T>::size_type pos ) const
    {
        return pos < this->size();
    }

    [[nodiscard]]
    //[[unsafe]]
```

```
constexpr reference get_reference( size_type pos ) noexcept
{
    return (*this)[pos];
}

[[nodiscard]]
//[[unsafe]]
constexpr const_reference get_reference( size_type pos ) const noexcept
{
    return (*this)[pos];
}
// at and operator[] replacements
[[nodiscard]]
//[[safe]]
constexpr value_type get_value( size_type pos, const std::source_location loca
{
    if(test(pos))
    {
        return get_reference(pos);
    }
    else
    {
        //throw std::out_of_range(to_string(pos, location));
        throw out_of_range_v2_1<size_type>(pos, location);
    }
}

template<class F>
[[nodiscard]]
//[[safe]]
constexpr auto transform_value( size_type pos, const F& transformer, const std
{
    if(test(pos))
    {
        return transformer(get_reference(pos));
    }
    else
    {
        //throw std::out_of_range(to_string(pos, location));
        throw out_of_range_v2_1<size_type>(pos, location);
    }
}

[[nodiscard]]
//[[safe]]
constexpr value_type get_value( size_type pos, const_reference default_value )
{
    if(test(pos))
    {
```

```
        return get_reference(pos);
    }
    else
    {
        return default_value;
    }
}

template<class F>
[[nodiscard]]
//[[safe]]
constexpr auto transform_value( size_type pos, const F& transformer, const aut
{
    if(test(pos))
    {
        return transformer(get_reference(pos));
    }
    else
    {
        return default_value;
    }
}

[[nodiscard]]
//[[safe]]
constexpr std::optional<value_type> get_optional( size_type pos ) const noexce
{
    if(test(pos))
    {
        return std::optional<value_type>{get_reference(pos)};
    }
    else
    {
        return std::nullopt;
    }
}

template<class F>
[[nodiscard]]
//[[safe]]
constexpr auto transform_optional( size_type pos, const F& transformer ) const
{
    if(test(pos))
    {
        return std::optional<decltype(transformer(get_reference(pos)))>{transf
    }
    else
    {
        return std::optional<decltype(transformer(get_reference(pos)))>{};
```

```
    }
}

[[nodiscard]]
//[[safe]]
constexpr value_type get_or_terminate( size_type pos, const std::source_locati
{
    if(test(pos))
    {
        return get_reference(pos);
    }
    else
    {
        log(pos, location);
        std::terminate();
    }
}

template<class F>
[[nodiscard]]
//[[safe]]
constexpr auto transform_or_terminate( size_type pos, const F& transformer, co
{
    if(test(pos))
    {
        return transformer(get_reference(pos));
    }
    else
    {
        log(pos, location);
        std::terminate();
    }
}

//[[safe]]
constexpr void set_value( size_type pos, const_reference value, const std::sou
{
    if(test(pos))
    {
        get_reference(pos) = value;
    }
    else
    {
        //throw std::out_of_range(to_string(pos, location));
        throw out_of_range_v2_1<size_type>(pos, location);
    }
}

template<class F>
```

```
//[[safe]]
constexpr void visit_value( size_type pos, const F& copier, const std::source_
{
    if(test(pos))
    {
        copier(get_reference(pos));
    }
    else
    {
        //throw std::out_of_range(to_string(pos, location));
        throw out_of_range_v2_1<size_type>(pos, location);
    }
}

//[[safe]]
constexpr void set_or_terminate( size_type pos, const_reference value, const s
{
    if(test(pos))
    {
        get_reference(pos) = value;
    }
    else
    {
        log(pos, location);
        std::terminate();
    }
}

template<class F>
//[[safe]]
constexpr void visit_or_terminate( size_type pos, const F& copier, const std::
{
    if(test(pos))
    {
        copier(get_reference(pos));
    }
    else
    {
        log(pos, location);
        std::terminate();
    }
}

// https://en.wikipedia.org/wiki/Cropping_(image)
//[[safe]]
constexpr void set_and_crop( size_type pos, const_reference value ) noexcept
{
    if(test(pos))
    {
```

```
        get_reference(pos) = value;
    }
}

template<class F>
constexpr void visit_and_crop( size_type pos, const F& copier ) noexcept
{
    if(test(pos))
    {
        copier(get_reference(pos));
    }
}

//[[safe]]
constexpr void set_and_grow( size_type pos, const_reference value )
{
    this->resize(std::max(this->size(), pos + 1));
    get_reference(pos) = value;
}

template<class F>
//[[safe]]
constexpr void visit_and_grow( size_type pos, const F& copier )
{
    this->resize(std::max(this->size(), pos + 1));
    copier(get_reference(pos));
}

//[[safe]]
constexpr void set_and_grow( size_type pos, const_reference value, const_refer
{
    this->resize(std::max(this->size(), pos + 1), default_value);
    get_reference(pos) = value;
}

template<class F>
//[[safe]]
constexpr void visit_and_grow( size_type pos, const F& copier, const_reference
{
    this->resize(std::max(this->size(), pos + 1), default_value);
    copier(get_reference(pos));
}
// front replacements
[[nodiscard]]
//[[safe]]
constexpr value_type get_front_value( const std::source_location location = st
{
    if(this->empty())
    {
```

```
        //throw std::out_of_range(to_string(0, location));
        throw out_of_range_v2_1<size_type>(0, location);
    }
    else
    {
        return get_reference(0);
    }
}

template<class F>
[[nodiscard]]
//[[safe]]
constexpr auto transform_front_value( const F& transformer, const std::source_
{
    if(this->empty())
    {
        //throw std::out_of_range(to_string(0, location));
        throw out_of_range_v2_1<size_type>(0, location);
    }
    else
    {
        return transformer(get_reference(0));
    }
}

[[nodiscard]]
//[[safe]]
constexpr value_type get_front_value( const_reference default_value ) const no
{
    if(this->empty())
    {
        return default_value;
    }
    else
    {
        return get_reference(0);
    }
}

template<class F>
[[nodiscard]]
//[[safe]]
constexpr auto transform_front_value( const F& transformer, const auto& defaul
{
    if(this->empty())
    {
        return default_value;
    }
    else
```

```
    {
        return transformer(get_reference(0));
    }
}

[[nodiscard]]
//[[safe]]
constexpr std::optional<value_type> get_front_optional() const noexcept
{
    if(this->empty())
    {
        return std::nullopt;
    }
    else
    {
        return std::optional<value_type>{get_reference(0)};
    }
}

template<class F>
[[nodiscard]]
//[[safe]]
constexpr auto transform_front_optional( const F& transformer ) const noexcept
{
    if(this->empty())
    {
        return std::optional<decltype(transformer(get_reference(0)))>{};
    }
    else
    {
        return std::optional<decltype(transformer(get_reference(0)))>{transfor
    }
}

[[nodiscard]]
//[[safe]]
constexpr value_type get_front_or_terminate( const std::source_location locati
{
    if(this->empty())
    {
        log(0, location);
        std::terminate();
    }
    else
    {
        return get_reference(0);
    }
}
```

```
template<class F>
[[nodiscard]]
//[[safe]]
constexpr auto transform_front_or_terminate( const F& transformer, const std::
{
    if(this->empty())
    {
        log(0, location);
        std::terminate();
    }
    else
    {
        return transformer(get_reference(0));
    }
}

//[[safe]]
constexpr void set_front_value( const_reference value, const std::source_locat
{
    if(this->empty())
    {
        //throw std::out_of_range(to_string(0, location));
        throw out_of_range_v2_1<size_type>(0, location);
    }
    else
    {
        get_reference(0) = value;
    }
}

template<class F>
//[[safe]]
constexpr void visit_front_value( const F& copier, const std::source_location
{
    if(this->empty())
    {
        //throw std::out_of_range(to_string(0, location));
        throw out_of_range_v2_1<size_type>(0, location);
    }
    else
    {
        copier(get_reference(0));
    }
}

//[[safe]]
constexpr void set_front_or_terminate( const_reference value, const std::sourc
{
    if(this->empty())
```

```
    {
        log(0, location);
        std::terminate();
    }
    else
    {
        get_reference(0) = value;
    }
}

template<class F>
//[[safe]]
constexpr void visit_front_or_terminate( const F& copier, const std::source_lo
{
    if(this->empty())
    {
        log(0, location);
        std::terminate();
    }
    else
    {
        copier(get_reference(0));
    }
}

// https://en.wikipedia.org/wiki/Cropping_(image)
//[[safe]]
constexpr void set_front_and_crop( const_reference value ) noexcept
{
    if(!this->empty())
    {
        get_reference(0) = value;
    }
}

template<class F>
constexpr void visit_front_and_crop( const F& copier ) noexcept
{
    if(!this->empty())
    {
        copier(get_reference(0));
    }
}

//[[safe]]
constexpr void set_front_and_grow( const_reference value )
{
    this->resize(std::max(this->size(), 1ul));
    get_reference(0) = value;
```

```
}

template<class F>
//[[safe]]
constexpr void visit_front_and_grow( const F& copier )
{
    this->resize(std::max(this->size(), 1ul));
    copier(get_reference(0));
}

//[[safe]]
constexpr void set_front_and_grow( const_reference value, const_reference defa
{
    this->resize(std::max(this->size(), 1ul), default_value);
    get_reference(0) = value;
}

template<class F>
//[[safe]]
constexpr void visit_front_and_grow( const F& copier, const_reference default_
{
    this->resize(std::max(this->size(), 1ul), default_value);
    copier(get_reference(0));
}
// back replacements
[[nodiscard]]
//[[safe]]
constexpr value_type get_back_value( const std::source_location location = std
{
    if(this->empty())
    {
        //throw std::out_of_range(to_string(0, location));
        throw out_of_range_v2_1<size_type>(0, location);
    }
    else
    {
        return get_reference(this->size() - 1);
    }
}

template<class F>
[[nodiscard]]
//[[safe]]
constexpr auto transform_back_value( const F& transformer, const std::source_l
{
    if(this->empty())
    {
        //throw std::out_of_range(to_string(0, location));
        throw out_of_range_v2_1<size_type>(0, location);
```

```
    }
    else
    {
        return transformer(get_reference(this->size() - 1));
    }
}

[[nodiscard]]
//[[safe]]
constexpr value_type get_back_value( const_reference default_value ) const noe
{
    if(this->empty())
    {
        return default_value;
    }
    else
    {
        return get_reference(this->size() - 1);
    }
}

template<class F>
[[nodiscard]]
//[[safe]]
constexpr auto transform_back_value( const F& transformer, const auto& default
{
    if(this->empty())
    {
        return default_value;
    }
    else
    {
        return transformer(get_reference(this->size() - 1));
    }
}

[[nodiscard]]
//[[safe]]
constexpr std::optional<value_type> get_back_optional() const noexcept
{
    if(this->empty())
    {
        return std::nullopt;
    }
    else
    {
        return std::optional<value_type>{get_reference(this->size() - 1)};
    }
}
```

```
template<class F>
[[nodiscard]]
//[[safe]]
constexpr auto transform_back_optional( const F& transformer ) const noexcept
{
    if(this->empty())
    {
        return std::optional<decltype(transformer(get_reference(0)))>{};
    }
    else
    {
        return std::optional<decltype(transformer(get_reference(0)))>{transfor
    }
}

[[nodiscard]]
//[[safe]]
constexpr value_type get_back_or_terminate( const std::source_location locatio
{
    if(this->empty())
    {
        log(0, location);
        std::terminate();
    }
    else
    {
        return get_reference(this->size() - 1);
    }
}

template<class F>
[[nodiscard]]
//[[safe]]
constexpr auto transform_back_or_terminate( const F& transformer, const std::s
{
    if(this->empty())
    {
        log(0, location);
        std::terminate();
    }
    else
    {
        return transformer(get_reference(this->size() - 1));
    }
}

//[[safe]]
constexpr void set_back_value( const_reference value, const std::source_locati
```

```
{
    if(this->empty())
    {
        //throw std::out_of_range(to_string(0, location));
        throw out_of_range_v2_1<size_type>(0, location);
    }
    else
    {
        get_reference(this->size() - 1) = value;
    }
}

template<class F>
//[[safe]]
constexpr void visit_back_value( const F& copier, const std::source_location l
{
    if(this->empty())
    {
        //throw std::out_of_range(to_string(0, location));
        throw out_of_range_v2_1<size_type>(0, location);
    }
    else
    {
        copier(get_reference(this->size() - 1));
    }
}

//[[safe]]
constexpr void set_back_or_terminate( const_reference value, const std::source
{
    if(this->empty())
    {
        log(0, location);
        std::terminate();
    }
    else
    {
        get_reference(this->size() - 1) = value;
    }
}

template<class F>
//[[safe]]
constexpr void visit_back_or_terminate( const F& copier, const std::source_loc
{
    if(this->empty())
    {
        log(0, location);
        std::terminate();
```

```
    }
    else
    {
        copier(get_reference(this->size() - 1));
    }
}

// https://en.wikipedia.org/wiki/Cropping_(image)
//[[safe]]
constexpr void set_back_and_crop( const_reference value ) noexcept
{
    if(!this->empty())
    {
        get_reference(this->size() - 1) = value;
    }
}

template<class F>
constexpr void visit_back_and_crop( const F& copier ) noexcept
{
    if(!this->empty())
    {
        copier(get_reference(this->size() - 1));
    }
}

//[[safe]]
constexpr void set_back_and_grow( const_reference value )
{
    this->resize(std::max(this->size(), 1ul));
    get_reference(this->size() - 1) = value;
}

template<class F>
//[[safe]]
constexpr void visit_back_and_grow( const F& copier )
{
    this->resize(std::max(this->size(), 1ul));
    copier(get_reference(this->size() - 1));
}

//[[safe]]
constexpr void set_back_and_grow( const_reference value, const_reference defau
{
    this->resize(std::max(this->size(), 1ul), default_value);
    get_reference(this->size() - 1) = value;
}

template<class F>
```

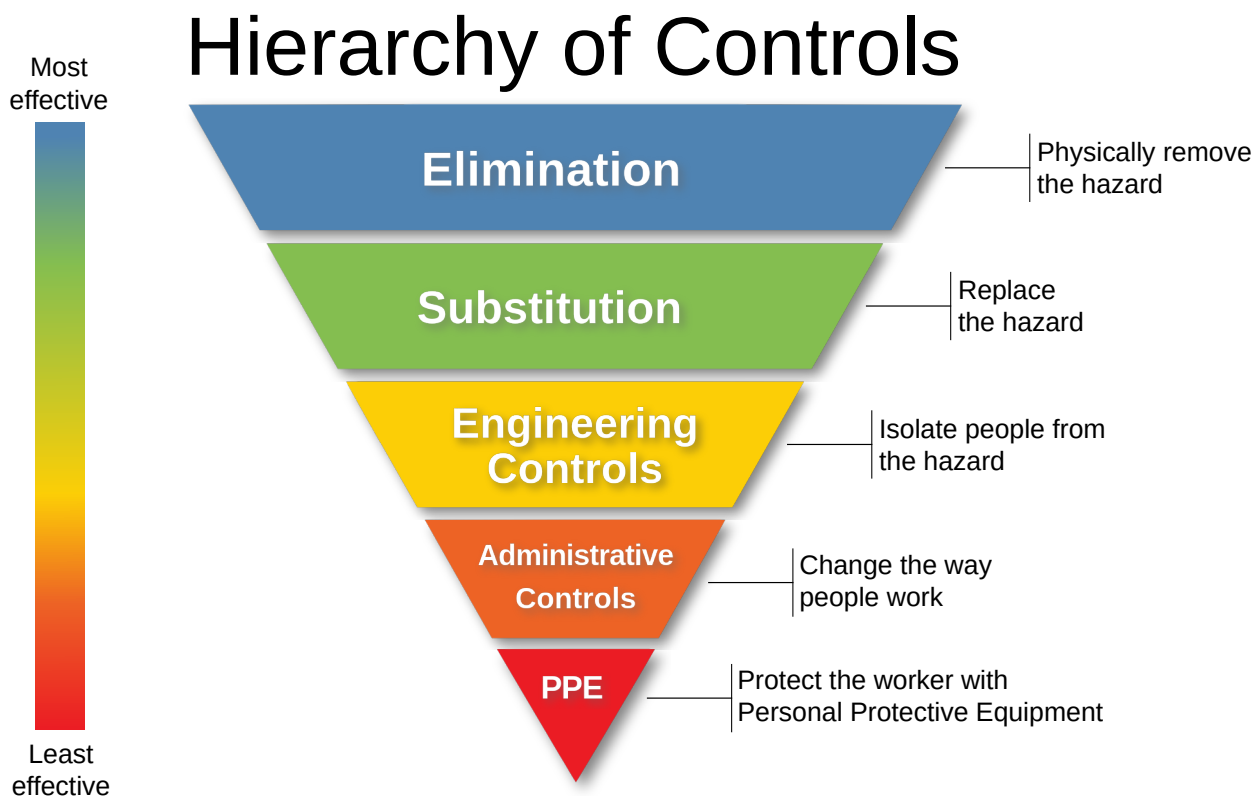
```
    //[[safe]]
    constexpr void visit_back_and_grow( const F& copier, const_reference default_v
    {
        this->resize(std::max(this->size(), 1ul), default_value);
        copier(get_reference(this->size() - 1));
    }
};

template<class T>
constexpr void copy_and_crop( std::span<T> destination, std::size_t pos, std::span
{
    std::size_t dsize = destination.size();
    if(pos < dsize)
    {
        std::size_t length = std::min(dsize - pos, source.size());
        std::copy_n(source.begin(), length, destination.begin() + pos);
    }
}

template<class T>
constexpr void copy_and_grow( std::vector<T>& destination, std::size_t pos, std::s
{
    std::size_t ssize = source.size();
    destination.resize(std::max(destination.size(), pos + ssize));
    std::copy_n(source.begin(), ssize, destination.begin() + pos);
}

// gcc, clang: -std=c++2b -O3
// msvc:       /std:c++20
// shadowing to safety
// shadowing enhances safety
int main()
{
    try
    {
        my_vector<int> myints{1};
        bool t = myints.test(0);
        t = myints.test(1);
        // get whole
        int pv = myints.get_value( 0/*5*/ );//throws
        pv = myints.get_value( 0, 42 );//default value
        std::optional<int> opv = myints.get_optional( 0 );
        pv = myints.get_or_terminate( 0 );

        pv = myints.get_front_value();//throws
        pv = myints.get_front_value( 42 );//default value
        opv = myints.get_front_optional();
        pv = myints.get_front_or_terminate();
```

```
pv = myints.get_back_value();//throws
pv = myints.get_back_value( 42 );//default value
opv = myints.get_back_optional();
pv = myints.get_back_or_terminate();
// set whole
myints.set_value( 0, 42 );//throws
myints.set_or_terminate( 0, 42 );
myints.set_and_crop( 0, 42 );
myints.set_and_grow( 1, 42 );
myints.set_and_grow( 3, 42, 0 );

myints.set_front_value( 42 );//throws
myints.set_front_or_terminate( 42 );
myints.set_front_and_crop( 42 );
myints.set_front_and_grow( 42 );
myints.set_front_and_grow( 42, 0 );

myints.set_back_value( 42 );//throws
myints.set_back_or_terminate( 42 );
myints.set_back_and_crop( 42 );
myints.set_back_and_grow( 42 );
myints.set_back_and_grow( 42, 0 );

my_vector<std::string> mystrings{"1"};
bool ts = mystrings.test(0);
ts = mystrings.test(1);

auto s42 = "42"s;
// get/transform whole or part
size_t pvi = mystrings.transform_value( 0, [](const std::string& item){ret
pvi = mystrings.transform_value( 0, [](const std::string& item){return ite
std::optional<size_t> opvs = mystrings.transform_optional( 0, [](const std
pvi = mystrings.transform_or_terminate( 0, [](const std::string& item){ret

pvi = mystrings.transform_front_value( [](const std::string& item){return
pvi = mystrings.transform_front_value( [](const std::string& item){return
opvs = mystrings.transform_front_optional( [](const std::string& item){ret
pvi = mystrings.transform_front_or_terminate( [](const std::string& item){

pvi = mystrings.transform_back_value( [](const std::string& item){return i
pvi = mystrings.transform_back_value( [](const std::string& item){return i
opvs = mystrings.transform_back_optional( [](const std::string& item){retu
pvi = mystrings.transform_back_or_terminate( [](const std::string& item){r
// set/visit whole or part
mystrings.visit_value( 0, [&s42](std::string& item){item = s42;} );
mystrings.visit_or_terminate( 0, [&s42](std::string& item){item = s42;} );
mystrings.visit_and_crop( 0, [&s42](std::string& item){item = s42;} );
mystrings.visit_and_grow( 1, [&s42](std::string& item){item = s42;} );
mystrings.visit_and_grow( 3, [&s42](std::string& item){item = s42;}, "0" )
```

```
        mystrings.visit_front_value( [&s42](std::string& item){item = s42;} );
        mystrings.visit_front_or_terminate( [&s42](std::string& item){item = s42;}
        mystrings.visit_front_and_crop( [&s42](std::string& item){item = s42;} );
        mystrings.visit_front_and_grow( [&s42](std::string& item){item = s42;} );
        mystrings.visit_front_and_grow( [&s42](std::string& item){item = s42;}, "0

        mystrings.visit_back_value( [&s42](std::string& item){item = s42;} );
        mystrings.visit_back_or_terminate( [&s42](std::string& item){item = s42;}
        mystrings.visit_back_and_crop( [&s42](std::string& item){item = s42;} );
        mystrings.visit_back_and_grow( [&s42](std::string& item){item = s42;} );
        mystrings.visit_back_and_grow( [&s42](std::string& item){item = s42;}, "0"

        std::vector<std::string> destination{"1", "2", "3"};
        /*const*/ std::vector<std::string> source1{"3", "4", "5"};
        /*const*/ std::vector<std::string> source2{"6", "7", "8"};

        copy_and_crop(std::span<std::string>{destination}, 1, std::span<const std:
        copy_and_grow(destination, 2, std::span<const std::string>{source2});// 1,
        //static_assert( destination[0] == "1"s );
        //static_assert( destination[1] == "3"s );
        //static_assert( destination[2] == "6"s );
        //static_assert( destination[3] == "7"s );
        //static_assert( destination[4] == "8"s );
        assert( destination.size() == 5 );
        assert( destination[0] == "1"s );
        assert( destination[1] == "3"s );
        assert( destination[2] == "6"s );
        assert( destination[3] == "7"s );
        assert( destination[4] == "8"s );
    }
    //catch(const std::out_of_range& oor)
    //{
    //    std::clog << oor.what() << '\n';
    //}
    catch(const out_of_range_v2_1<std::vector<int>::size_type>& oor)
    {
        std::stringstream ss;
        oor.what(ss);
        std::clog << ss.str() << '\n';
    }
    return 0;
}
```

Resolution

How effective is the safety proposed? Let's look at the *[exponential]* safety scale [4].



While this proposal does greatly increase safety by increasing the number of safe element access functions from 0 to 56, based on this scale, the effectiveness is limited because it is at best `PPE`, personal protective equipment that only works if used. While **NOT** a part of the proposal, throughout this proposal you may have noticed the following description attributes.

```
[[unsafe(reason=["range", "dangling_reference", "reference_invalidation"])]]
[[unsafe(reason=["dangling_reference", "reference_invalidation"])]]
[[safe]]
```

Armed with this type of documentation, a tool could be provided that outputs a JSON/YAML statistical summary report representing an audit of the safety of the code. This report could be checked in with the source code so that a automated code reviewer could reject code submission if the patch would render the code base less safe or proportionally so. Further, those code who can't have any unsafety, such as new code, would benefit from static analysis, preferably built into the compiler, that would reject any deprecated and documented unsafe code especially when a substitute has been provided. These are high level tools used by owners, architects, managers, team leads and auditors on the project/module level.

Summary

Excluding overloads and contrasting the current `operator[]`, `at`, `back` and `front` methods with this proposals value based additions, the current design is 75% undefined/unsafe for range based access on `std::array`, `std::deque`, `std::string`, `std::string_view` and `std::vector`. This grows to 100% for `std::span` since it currently doesn't have an `at` method. This doesn't even include the undefined/unsafe behavior of iterators, direct access via `data` or the fact that `operator[]` is disproportionately used over `at`, even in large code bases, which means, in the wild, this percentage is closer to 100%. This proposal adds 44 safe functions and 1 unsafe function with defined behavior. This drops the percentage of undefined/unsafe behavior for range based access to <11%. Consequently, with minimal training, it will be easier for a programmer to reach for a safer function over a less safe one. This doesn't even include the fact that all of the existing element access methods are also susceptible to dangling and invalidation issues which these new functions avoids and minimizes.

The advantages of adopting said proposal are as follows:

1. Reduces range based access errors
2. Reduces dangling references because there are a lower ratio of reference returning functions
3. Reduces reference invalidation errors because there are a lower ratio of reference returning functions
4. Allows programmers to avoid superfluous dynamic allocation when working with exceptions

Frequently Asked Questions

Why not just wait for contracts?

1. Besides the fact that we do not know exactly when `contracts` will land, real harm is occuring in the mean time.
2. Even if `contracts` were added to `operator[]`, `front` and `back`, these methods would still be returning references which would still be susceptible to dangling and invalidation issues.
3. The following void returning, crop, grow and batch funtions has utility apart from `contracts` because no error is expected, rather just well designed behavior.
   - `set_and_crop`
   - `set_front_and_crop`
   - `set_back_and_crop`

    - `set_and_grow`
    - `set_front_and_grow`
    - `set_back_and_grow`
    - `visit_and_crop`
    - `visit_front_and_crop`
    - `visit_back_and_crop`
    - `visit_and_grow`
    - `visit_front_and_grow`
    - `visit_back_and_grow`
    - `copy_and_crop`
    - `copy_and_grow`
4. Programmers like to be in control of their error handling strategy, (return, throw, terminate, void/crop, void/grow), which can vary from one call to the next.

References

1. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p0847r7.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p0847r7.html) ↩

2. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2011r1.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2011r1.html) ↩

3. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2672r0.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2672r0.html) ↩

4. https://www.cdc.gov/niosh/topics/hierarchy/default.html (https://www.cdc.gov/niosh/topics/hierarchy/default.html) ↩